

Paradigmi di Programmazione e Astrazione

Metodi Avanzati di Programmazione

Laurea Triennale in Informatica

Università degli Studi di Bari Aldo Moro

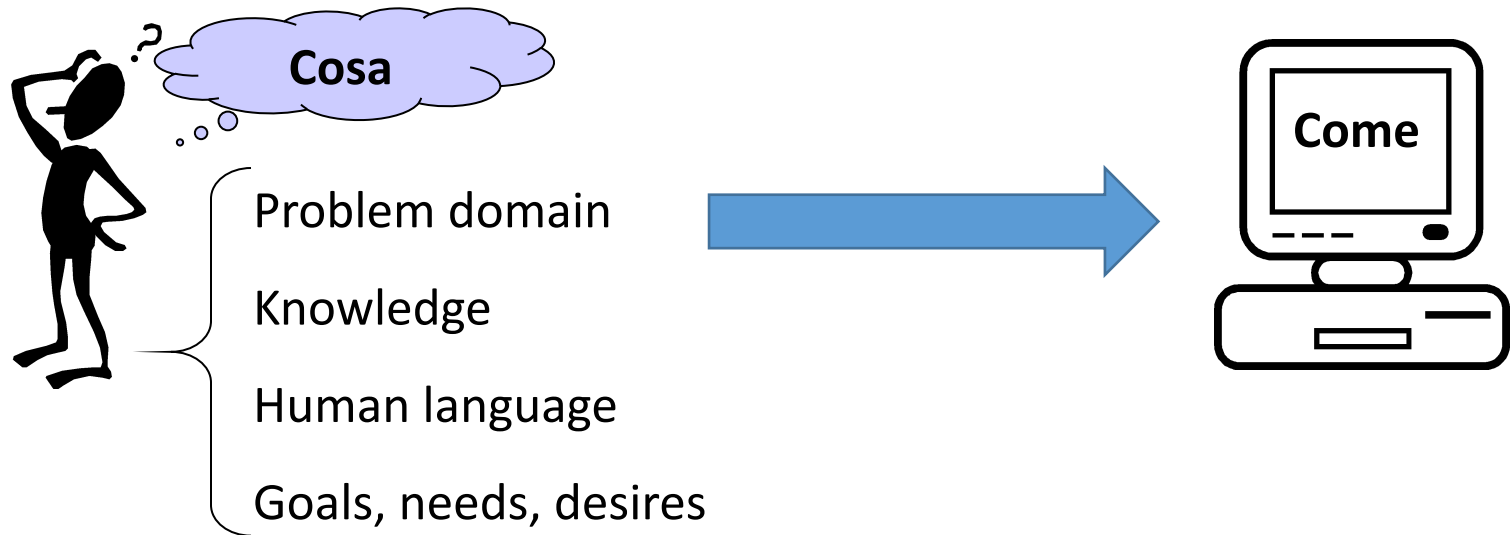
Docente: Pierpaolo Basile

**“A computer will do what you tell it to do, but that may be
much different from what you had in mind”**

Joseph Weizenbaum

Time

Dal “cosa” al “come” ...

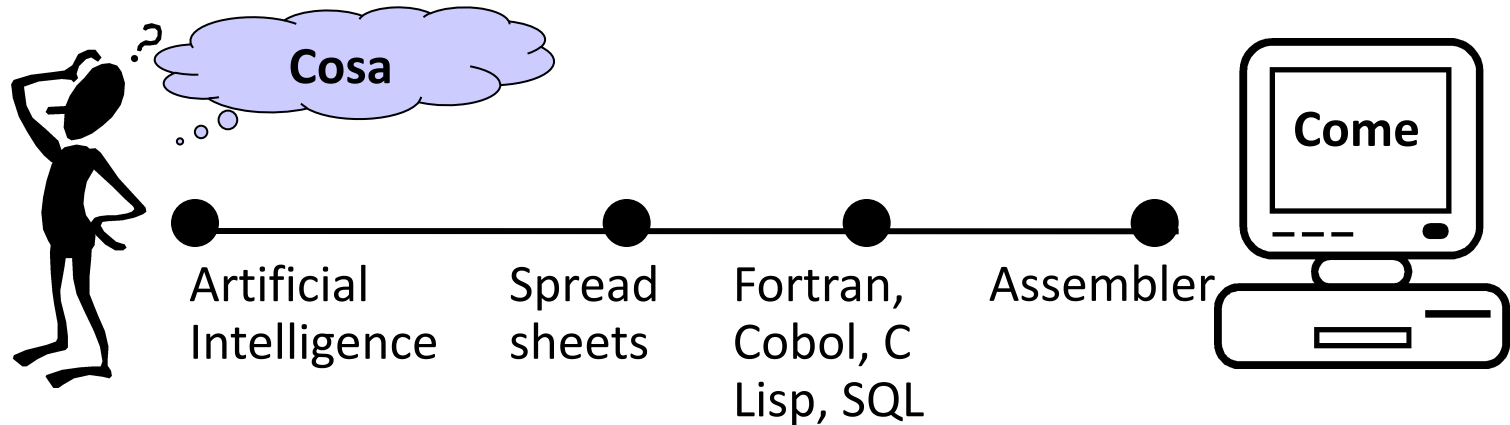


«La scienza del software studia il passaggio dal “**cosa**” al “**come**”»

E.A. Feigenbaum

Communications of the ACM, Maggio 1996

Dal “cosa” al “come” ...



L'estremo “cosa” (**what**) raccoglie gli obiettivi, i desideri e i bisogni dell'utente, la sua conoscenza del dominio del problema, tutti espressi in linguaggio naturale.

L'estremo “come” (**how**) è l'hardware che può raggiungere quegli obiettivi e soddisfare i bisogni e i desideri in modo estremamente procedurale e rigorosamente sequenziale.

... Dal “cosa” al “come”

La storia dell'informatica è stato un lento ma costante attraversamento dello spettro **what-how**.

La scienza della programmazione ha esplorato molti punti dello spettro, sempre combattuta fra il “cosa” e il “come”.

Ne sono nati diversi **paradigmi di programmazione**, ovvero *collezioni di modelli concettuali che insieme plasmano il processo di analisi, progettazione e sviluppo di un programma*.

Paradigma: complesso di regole metodologiche, modelli esplicativi, criteri di soluzione di problemi che caratterizza una comunità di scienziati in una fase determinata dell'evoluzione storica della loro disciplina

I paradigmi di programmazione

Questi modelli concettuali “strutturano” il pensiero in quanto determinano la forma di programmi validi.

Essi influenzano il modo in cui pensiamo e formuliamo le soluzioni, arrivando a condizionare perfino la possibilità di trovare una soluzione.



Per calcolare il fattoriale di n moltiplica 1 per 2, poi per 3, ... fino ad arrivare a n

Il fattoriale di n
è n per il
fattoriale di $n-1$



I paradigmi di programmazione

Un paradigma cambia fundamentalmente il modo in cui guardiamo ai problemi incontrati nel passato.

Esso ci deve dare un nuovo schema per pensare ai problemi futuri.

Esso cambia le nostre priorità, le nostre idee su quanto merita attenzione e su cosa sia importante.

Come nasce un paradigma?

Un nuovo paradigma è spesso **introdotto per risolvere un particolare problema**, ma si rivela poi adatto per altri.

L'idea di oggetto che incorpora dati e metodi per lavorare sui dati venne introdotta per la prima volta in SIMULA (K. Nygaard & O.J. Dahl), un linguaggio pensato per la simulazione.

È stata subito ripresa in altri linguaggi di programmazione, come Smalltalk, e approfondita tanto da divenire il cardine del paradigma object-oriented, oggi adottato in molti altri campi diversi da quello della simulazione.

Rapporto Paradigma-Linguaggi

Nel senso della macchina di Turing, **tutti i linguaggi di programmazione più comuni sono universali.**

Tuttavia ogni linguaggio di programmazione si basa, o meglio **supporta**, un particolare paradigma, fornendo:

- a) Le **primitive** di quel paradigma
- b) I **metodi di composizione** di quel paradigma
- c) Un **linguaggio utente appropriato** che rende chiari i programmi scritti secondo quel paradigma
- d) Un'esecuzione **efficiente** dei programmi scritti nel linguaggio scelto al punto c)

Rapporto Paradigma-Linguaggi

I linguaggi di programmazione sono, dunque, dotati di opportuni *costrutti linguistici* che riflettono i modelli concettuali di un paradigma, al fine di facilitare l'espressione di una soluzione definita attraverso i modelli concettuali del paradigma.

In realtà, i linguaggi di programmazione possono supportare *più di un paradigma*.

Categorie di paradigmi

1. Paradigmi che supportano tecniche di programmazione di basso livello
2. Paradigmi che supportano metodi di progettazione di algoritmi (e.g., divide-et-impera, programmazione dinamica)
3. Paradigmi che supportano gli approcci di alto livello alla programmazione (e.g., paradigma funzionale e quello basato su regole)

FOCUS del corso: **paradigmi del terzo tipo**

Approccio dei paradigmi al problem solving

I paradigmi che supportano la programmazione ad alto livello possono essere raggruppati in base al loro approccio alla soluzione dei problemi.

- a) Approccio operativo*: descrive per passi COME costruire una soluzione.
- b) Approccio dimostrazionale*: è una variante del precedente che illustra la soluzione in modo operativo per esempi specifici e lascia al sistema il compito di generalizzare queste soluzioni di esempi ad altri casi.
- c) Approccio definizionale*: esso stabilisce delle proprietà della soluzione in modo da vincolarla senza per questo descrivere come viene calcolata.

Approccio operativo

- L'algoritmo è visto come una "Ricetta"
- È il pilastro della programmazione imperativa
 - a) qui il programmatore agisce come un coordinatore che impartisce ordini sequenziali. Il focus è sul mutamento di stato del sistema attraverso istruzioni precise
- Esempi tipici: C, Pascal, Assembly
- Concetto chiave: il "flusso di controllo" (cicli for, istruzioni if, assegnazione di variabili)
- **Punto chiave: il programmatore controlla il processo**

Approccio dimostrazionale

- È una variante dell'operazionale, ma più “guidata dagli esempi”
- Si forniscono esempi concreti di input/output
- Il sistema generalizza automaticamente la soluzione. È tipico di contesti in cui si “insegna” al sistema cosa fare
- Esempi:
 - a) Programmazione per esempi
 - b) Machine learning (in senso lato)
- **Punto chiave: non scrivi l'algoritmo, mostri cosa deve fare**

Approccio definizionale

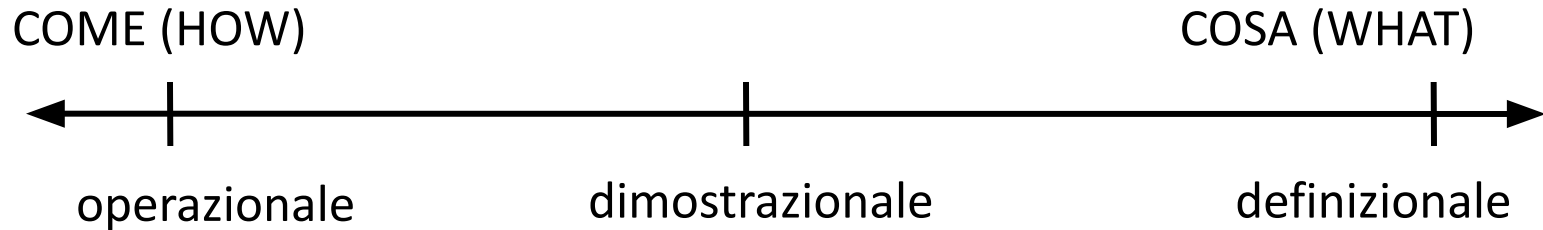
- Qui cambia completamente la prospettiva
- Si descrive COSA deve essere vero, non come ottenerlo
- Si specificano proprietà, vincoli o relazioni
- Il sistema trova autonomamente una soluzione che le soddisfi
- Esempi:
 - a) Programmazione logica (Prolog)
 - b) Programmazione dichiarativa
 - c) Programmazione funzionale (definizionale)

Punto chiave: descrivi il risultato, non il procedimento

Paradigmi (recap)

Approccio	Concetto chiave	Focus	Analogia
Operazionale	Ecco le istruzioni che devi seguire	Sequenza di passi	Seguire una ricetta passo-passo
Dimostrazionale	Guarda come si fa	Esempi specifici	Mostrare a qualcuno come si prepara una ricetta
Definizionale	Leggi cosa voglio ottenere	Proprietà e vincoli	Ordinare un piatto al ristorante, descrivere cosa vuoi non ti interessa il come

Approccio dei paradigmi al problem solving



Cosa deve indagare la scienza della programmazione?

- I *metodi di analisi dei problemi*, che consentono di passare da formulazioni spesso imprecise ed ambigue di un problema, a specifiche espresse in un qualche linguaggio formale.
- Le *tecniche di trasformazione dei programmi*, o tecniche di programmazione, che trasformano la specifica di un problema in un programma che lo risolve.
- I *metodi di verifica della correttezza*, che affrontano il problema di verificare la terminazione dei programmi e la rispondenza alle specifiche date.

Cosa deve indagare la scienza della programmazione?

- La *teoria della complessità computazionale*, che studia l'efficienza delle soluzioni trovate.
- La *teoria dei linguaggi formali e traduttori*, che studia gli strumenti utilizzabili per comunicare un algoritmo ad un elaboratore.

Quindi le *problematiche* suddette rappresentano una dimensione rispetto alla quale classificare gli studi nel campo della programmazione.

Perché studiare diversi paradigmi?

- Nessun singolo paradigma è appropriato per tutti i problemi. Le applicazioni complesse necessitano l'adozione di più paradigmi di programmazione.
- Ogni singolo paradigma di programmazione è caratterizzato da:
 - a) Un diverso *potere di elisione*, cioè capacità di esprimere qualcosa in modo conciso
 - b) Una diversa *invarianza rispetto ai cambiamenti* apportati nella strategia di soluzione di un problema

Approccio multi-paradigmatico: quali difficoltà?

Occorre sempre bilanciare i costi dovuti all'uniformità di paradigma con i costi determinati dall'uso di diversi paradigmi per un medesimo problema.

I **costi** sono

- Costi iniziali di apprendimento
- Costi di debugging
- Costi di variazione dovuti all'evoluzione del programma
- Costi di esecuzione dell'applicazione

Quale classe di applicazioni software?

Le applicazioni software possono essere classificate rispetto alla natura degli elementi che sono di interesse primario nell'applicazione:

- ***Applicazioni orientate alla realizzazione di funzioni***: la complessità prevalente riguarda le funzioni da realizzare.
- ***Applicazioni orientate alla gestione dei dati***: l'aspetto prevalente è rappresentato dai dati che vengono memorizzati, ricercati e modificati, e che costituiscono il patrimonio informativo di una organizzazione.
- ***Applicazioni orientate al controllo***: la complessità prevalente del sistema riguarda il controllo delle attività che si sincronizzano e cooperano durante l'evoluzione del sistema.

Quale classe di applicazioni software?

Rispetto al flusso di esecuzione delle attività:

- **Applicazioni sequenziali**: sono caratterizzate da un unico flusso di controllo che governa l'evoluzione dell'applicazione. Queste sono le applicazioni più tradizionali, e vengono spesso adottate come riferimento per i metodi e le tecniche di base per la progettazione.
- **Applicazioni concorrenti**: sono caratterizzate dal fatto che le varie attività che compongono il sistema non hanno una natura inerentemente sequenziale, ma necessitano di meccanismi di sincronizzazione e comunicazione.
- **Applicazioni dipendenti dal tempo**: sono influenzate da vincoli temporali riguardanti sia la velocità di esecuzione delle attività sia la necessità di sincronizzare le attività stesse.

Di quali applicazioni software ci occupiamo nel corso?

Appl. orientate
alla realizzaz. di
funzioni

Editor di
testo

Elaborazione
veloce di
immagini

Appl. orientate
alla gestione dei
dati

Sistema
informativo
biblioteca

Sistema
informativo
bancario

Applicazioni
orientate al
controllo

Sistema di
emergenza
sanitaria

Sequenziali

Concorrenti

Dipendenti dal tempo

Paradigmi operazionali

Sono caratterizzati da sequenze computazionali passo-passo.

Problema: è difficile stabilire se l'insieme dei valori calcolati operativamente è proprio l'insieme dei valori soluzione.

Le tecniche di *verifica* e di *debugging* cercano di superare questo problema di programmazione.

Spesso ci si accontenta di stabilire che i due insiemi siano “*sufficientemente vicini*”, ovvero indistinguibili per la sottoclasse *attesa* di problemi effettivi.

Side-effecting

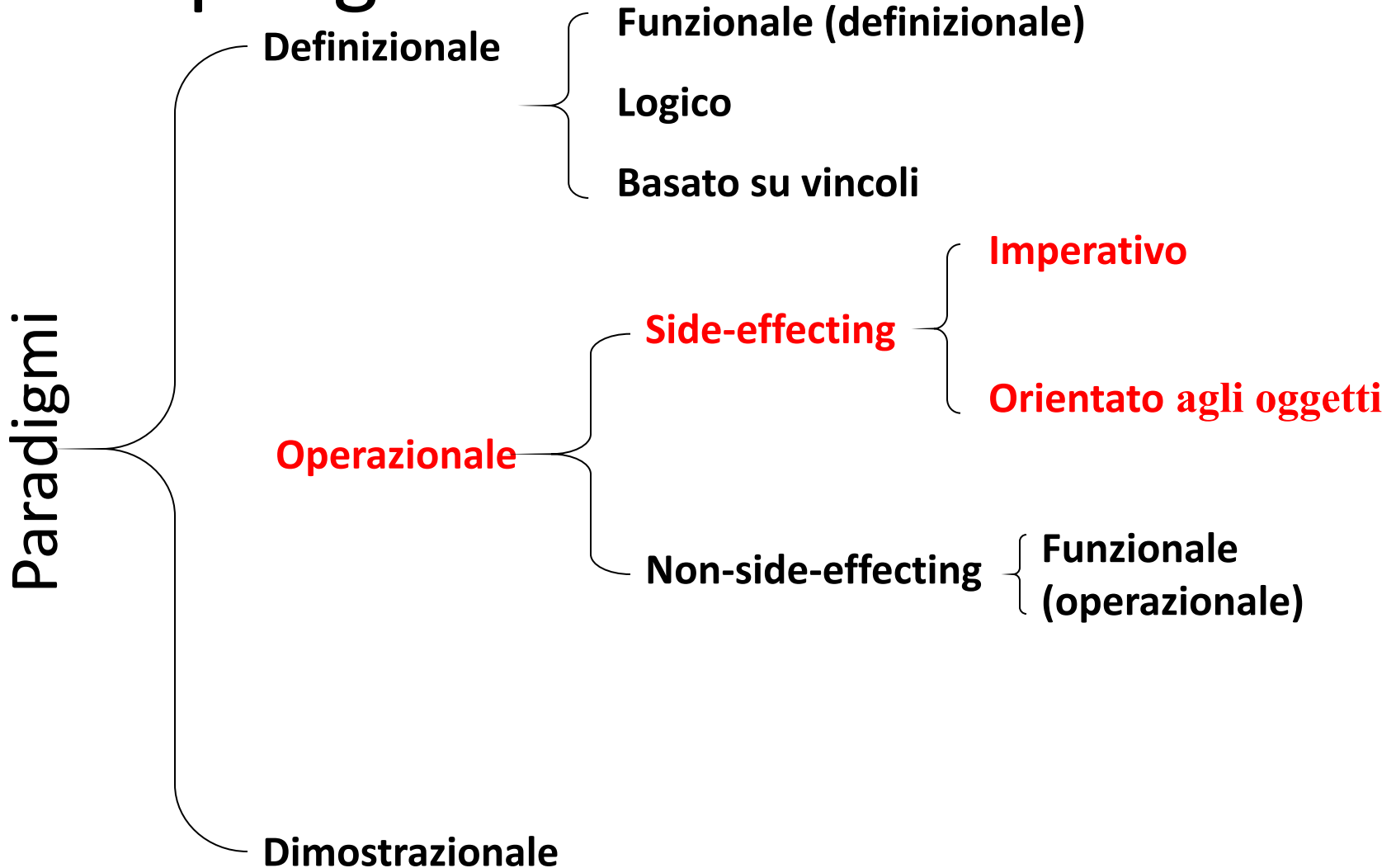
I paradigmi operazionali si distinguono in:

- ***Side-effecting***: procedono modificando ripetutamente la loro rappresentazione dei dati (le variabili sono legate a locazioni di memoria)
- ***Non-side-effecting***: procedono creando continuamente nuovi dati. Questi paradigmi includono quelli che tradizionalmente sono detti *funzionali*.

I paradigmi operazionali “***side-effecting***” si distinguono in:

- a) Imperativi
- b) Orientati agli oggetti

Riepilogando ...



Sequenziale vs. Concorrente

Per ognuno dei paradigmi visti precedentemente possiamo distinguere due versioni:

- *Sequenziale*: il flusso di controllo è unico
- *Concorrente* o *parallelo*: più flussi di controllo sono ammessi.

Nel corso verranno spiegati prevalentemente concetti legati alla versione *sequenziale*.

Riferimenti bibliografici

Per approfondimenti su una categorizzazione dei paradigmi di programmazione si raccomanda la lettura del seguente articolo:

- A.L. Ambler, M.H. Burnett, & B.A. Zimmerman. Operational Versus Definitional: A Perspective on Programming Paradigms. *IEEE Computer*, 25(9): 28-43, September 1992.

Astrazione nella progettazione

L'astrazione

Astrarre: dal lat. *abstrahere* = 'trascinare via', dal verbo *trahere* = trascinare.

Cosa trascinare via? Un concetto, una idea, un principio da una realtà concreta.

In ambito scientifico, *astrarre significa cambiare la rappresentazione di un problema.*

Perché astrarre?

L'obiettivo del cambio di rappresentazione è quello di concentrarsi su aspetti rilevanti dimenticando gli elementi secondari

L'astrazione

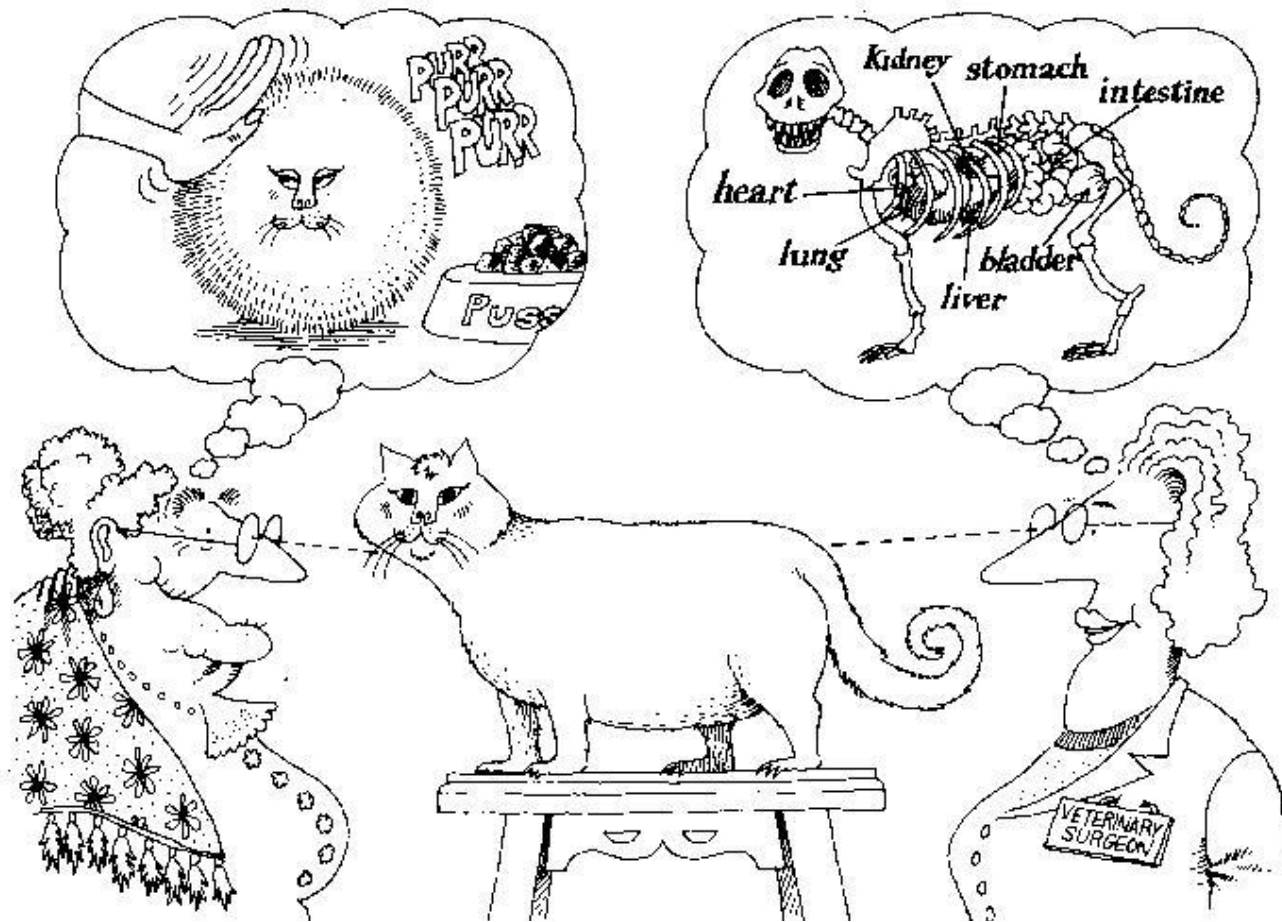
Non si tratta di **omettere** parti della rappresentazione di un problema, ma di **reformulare** lo stesso concentrando l'attenzione su idee generali piuttosto che su manifestazioni specifiche di quelle idee, tenendo conto della **prospettiva** di un osservatore.

Problema: trovare il percorso migliore per un commesso viaggiatore che deve visitare diverse città.

Se 'migliore' significa 'più breve', possiamo trascurare fattori come presenza di traffico e qualità della viabilità, e riformulare il problema come ricerca di minimo percorso in un grafo.



L'astrazione



L'astrazione si focalizza sulle caratteristiche essenziali di un oggetto, rispetto alla **prospettiva** di colui che osserva.

Astrazione: processo o entità

Il termine astrazione sotto-intende

- **Un processo**: l'estrazione delle informazioni essenziali e rilevanti per un particolare scopo, ignorando il resto dell'informazione
- **Una entità**: una descrizione semplificata di un sistema che enfatizza alcuni dei dettagli o proprietà trascurandone altri

Entrambe le viste sono valide e di fatto necessarie.

L'astrazione nell'analisi dei sistemi

Nell'**analisi dei sistemi** un'astrazione è una descrizione semplificata del sistema, che enfatizza alcuni dettagli o proprietà essenziali del sistema mentre ne ignora altri estranei.

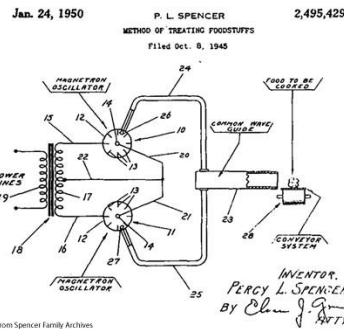
Esempio: controllo del traffico aereo

- Essenziale**: posizione del velivolo, velocità, etc.
- Irrilevante**: colore, nomi dei passeggeri, etc.

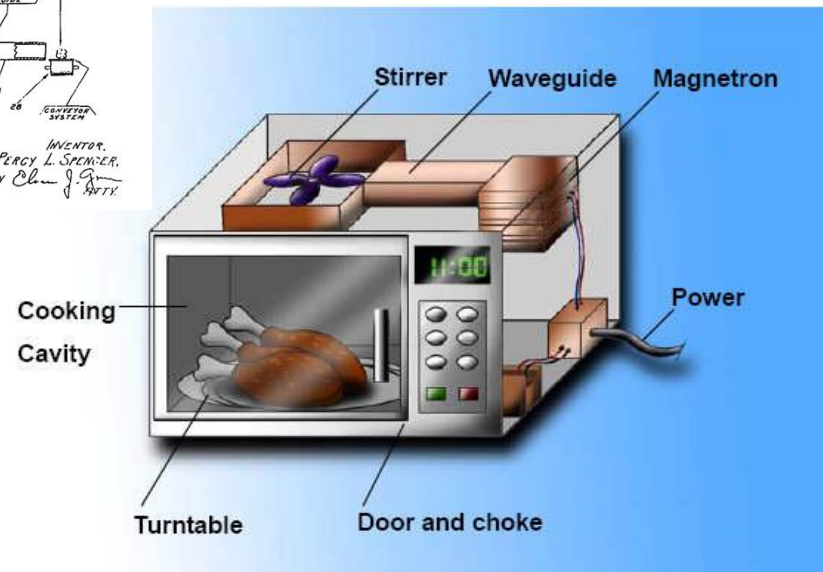
L'astrazione nell'analisi dei sistemi

Nel quotidiano il principio di astrazione è costantemente applicato ogni qualvolta utilizziamo uno strumento senza per questo sapere come è realizzato.

Esempio: utilizzo di un forno a microonde senza conoscerne la realizzazione.



brevetto



Astrazione e software

- Similmente, nella **programmazione** l'astrazione allude alla distinzione che si fa tra:

- cosa (*what*) fa un pezzo di codice
- come (*how*) esso è implementato

Per l'utente l'essenziale è cosa fa il codice mentre non è interessato ai dettagli della implementazione.

Astrazione e software

I sistemi software diventano sempre più complessi.

Per **padroneggiare la complessità** è necessario concentrarsi solo sui pochi aspetti che più interessano in un certo contesto ed ignorare i restanti.

L'**astrazione** permette ai progettisti di sistemi software di risolvere problemi complessi in modo **organizzato e gestibile**.

Astrazione funzionale

L'astrazione funzionale si riferisce alla **progettazione del software**, e in particolare alla possibilità di **specificare un modulo software che trasforma** dei dati di input in dati di output nascondendo i **dettagli algoritmici della trasformazione**.

- In sintesi:
 - Il modulo software deve trasformare un input in un output, cioè deve calcolare una funzione
 - I dettagli della trasformazione, cioè del calcolo, non sono visibili al consumatore (fruitore) del modulo.
 - Il consumatore conosce solo le corrette convenzioni di chiamata (**specifica sintattica**) e 'cosa' fa il modulo (**specifica semantica**).
 - Il consumatore deve fidarsi del risultato.

Astrazione funzionale

Esempio: modulo che realizza un operatore per il calcolo del fattoriale.

La *specificata sintattica* indica il nome del modulo (e.g., **fatt**), il tipo di dato in input (un intero) e il tipo di risultato (un intero), in modo da permettere la corretta chiamata del modulo. `fatt(intero) → intero`

La *specificata semantica* indica la trasformazione operata, cioè la funzione calcolata:

$$fatt(n) = \begin{cases} n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$$

Come la trasformazione è calcolata o **realizzata** (e.g., iterativamente o ricorsivamente) non è noto al fruitore del modulo.

Astrazione funzionale: specifiche semantiche

Problema: come specificare la semantica del modulo?

Un modo è quello di esprimere, mediante due **predicati**, la relazione che lega i dati di ingresso ai dati di uscita:

se il primo predicato (detto **precondizione**) è vero sui dati di ingresso e se il programma termina su quei dati, allora il secondo (detto **postcondizione**) è vero sui dati di uscita.

Queste specifiche semantiche sono dette **assiomatiche**.

Nell'esempio del fattoriale:

Precondizione: $n \in \mathbb{N}$

Postcondizione: $fatt(n) = \begin{cases} n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$

Astrazione funzionale: specifiche semantiche

Esercizio:

Un modulo ha in input un intero x e un vettore A di n interi e restituisce un intero p e un vettore B di n interi.

Siano date le seguenti:

Precondizione: $\{ n > 0 \wedge \forall i \in [1, n] A[i] \in \mathbb{Z} \}$

Postcondizione: $\{ \forall i \in [1, n] \exists j \in [1, n] B[i] = A[j] \wedge$
 $\forall i \in [1, p-1] B[i] \leq x \wedge \forall i \in [p+1, n] B[i] > x \}$

Riconoscete la funzione di questo modulo?

Stepwise refinement

L'astrazione funzionale si è affermata pienamente solo nei primi anni '70, quando emerse una metodologia che mirava a costruire i programmi progredendo dal generale al particolare (“**stepwise refinement**” o “**top-down programming**”).

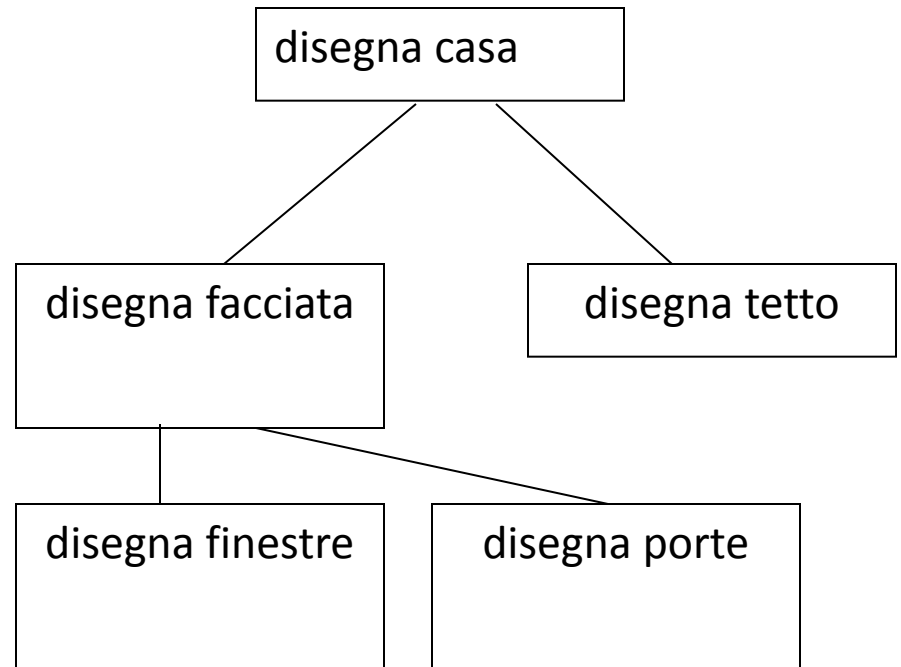
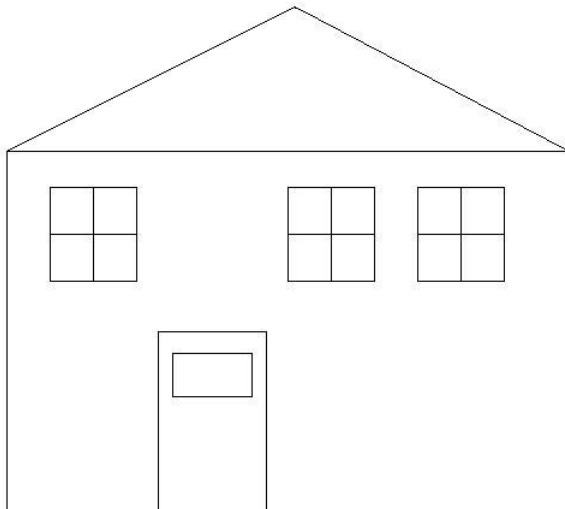
Secondo questa metodologia, per risolvere un problema (o affrontare un compito) P si procede come segue:

Stepwise refinement

1. Decomponi il compito P in sottocompiti $P_1, P_2, \dots, P_n$
2. Ipotizza di disporre di moduli $M_1, M_2, \dots, M_n$ che effettuano le trasformazioni richieste rispettivamente da $P_1, P_2, \dots, P_n$
3. Componi un modulo M che assolve al compito P usando i moduli $M_1, M_2, \dots, M_n$
4. Applica ricorsivamente la metodologia ai sottocompiti $P_1, P_2, \dots, P_n$ al fine di definire la realizzazione di $M_1, M_2, \dots, M_n$ fino a quando non si ottengono sottocompiti considerati elementari (o non ulteriormente decomponibili).

Stepwise refinement

Tipicamente la dipendenza fra moduli è rappresentata da un albero.



Stepwise refinement

Secondo la metodologia di stepwise refinement, il programmatore è libero di assumere l'esistenza di qualsiasi modulo (detto *stub*, lett. *matrice* di qualcosa, e.g., assegno) che si può applicare al particolare sottocompito e di cui fornisce una specifica, salvo dover poi specificare come quel modulo va realizzato.

L'astrazione funzionale è di supporto alla metodologia dello stepwise refinement.

Limiti dell'astrazione funzionale

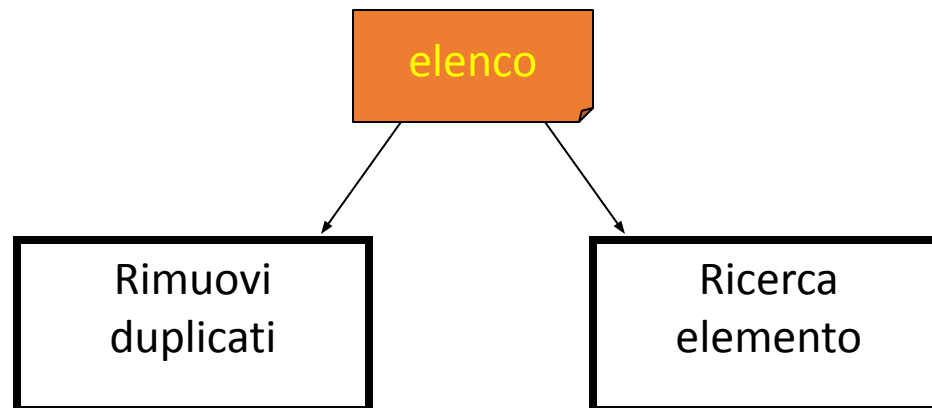
1. I dettagli relativi alla rappresentazione dei dati di input e output devono essere conosciuti da chi poi andrà a realizzare il modulo.

Esempio: un modulo che rimuove i duplicati in un elenco deve sapere se questo è realizzato con un array, un file, etc.

2. La rappresentazione è solitamente condivisa tra diversi moduli, per cui i cambiamenti apportati alla rappresentazione dei dati in input/output a un modulo si possono ripercuotere su molti altri moduli.

Limiti dell'astrazione funzionale

Esempio: due moduli operano su uno stesso elenco, uno per rimuovere duplicati e l'altro per ricercare un elemento. Per migliorare l'efficienza dell'ultimo sarebbe opportuno ricorrere a una rappresentazione dell'elenco con tabelle hash. Tuttavia ciò comporta un cambiamento nel primo modulo, che dovrà necessariamente operare la trasformazione (rimozione dei duplicati) in modo differente.



Limiti dell'astrazione funzionale

L'astrazione funzionale non permette di sviluppare soluzioni **invarianti ai cambiamenti nei dati** (sono invarianti solo ai cambiamenti nei processi di trasformazione che operano).

Questo rende difficoltosa la manutenzione delle soluzioni progettate.

È inappropriata per lo sviluppo di soluzioni a problemi complessi.

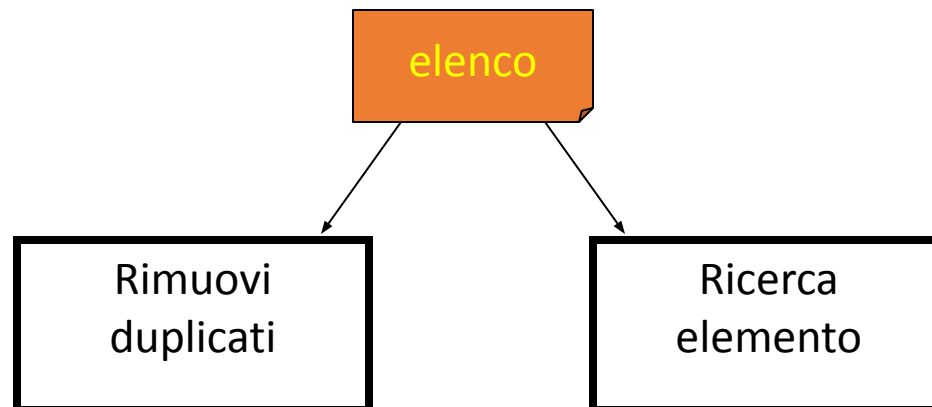
Astrazione dati

Alla base dell'**astrazione dati** c'è il principio che *non si può accedere direttamente alla rappresentazione di un dato, qualunque esso sia, ma solo attraverso un insieme di operazioni considerate lecite*
(**principio dell'astrazione dati**).

VANTAGGIO: un cambiamento nella rappresentazione del dato si ripercuote solo sulle operazioni lecite, che potrebbero subire delle modifiche, mentre non inficerà il codice che utilizza il dato astratto.

Astrazione dati

Esempio: Se i moduli 'Rimuovi duplicati' e 'Ricerca elemento' accedono all'elenco attraverso un insieme di operazioni lecite (e.g., 'dammi il prossimo elemento', 'non ci sono più elementi', ecc.) il cambiamento della rappresentazione dell'elenco richiede una riformulazione delle operazioni lecite, ma non influenzerà i due moduli.



Information Hiding

In generale un principio di astrazione suggerisce di **occultare l'informazione** (*information hiding*) sulla rappresentazione del dato

- sia perché non necessaria al fruitore dell'entità astratta
- sia perché la sua rivelazione creerebbe delle inutili dipendenze che comprometterebbero l'invarianza ai cambiamenti

Information Hiding

Il principio dell'astrazione funzionale suggerisce di occultare i dettagli del **processo di trasformazione** (“come” esso è operato).

Il principio dell'astrazione dati identifica nella **rappresentazione del dato** l'informazione da nascondere.

In entrambi i casi non si dice COME farlo.

Questo sarà chiarito quando approfondiremo il tema dell'astrazione nella programmazione.

Incapsulamento

L'**incapsulamento** (**encapsulation**) è una tecnica di progettazione consistente nell'impacchettare (o "racchiudere in capsule") una collezione di entità, creandone una barriera concettuale.

Come l'astrazione, l'incapsulamento sottointende

- *Un processo*: l'impacchettamento
- *Una entità*: il «pacchetto» ottenuto

Incapsulamento

Esempi

- una procedura impacchetta diversi comandi
- una libreria incapsula diverse funzioni
- un oggetto incapsula un dato e un insieme di operazioni sul dato

Incapsulamento

L'incapsulamento NON dice come devono essere le “pareti” del pacchetto (o capsula), che potranno essere:

- **Trasparenti**: permettendo di vedere **tutto** ciò che è stato impacchettato
- **Traslucide**: permettendo di **vedere** in modo **parziale** il contenuto
- **Opache**: **nascondendo tutto** il contenuto del pacchetto

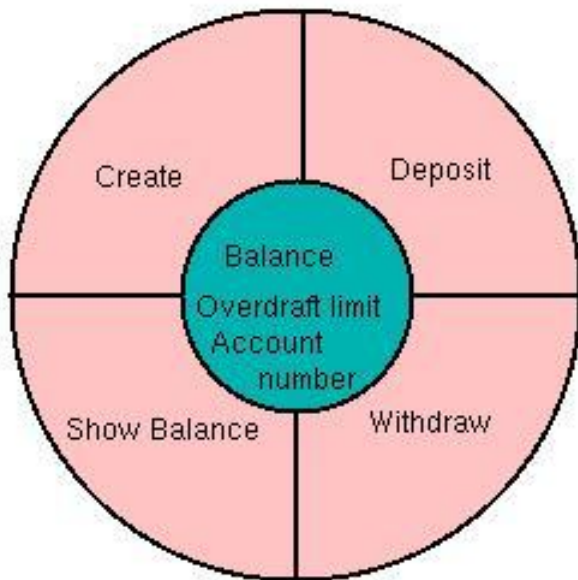
Astrazione dati & Incapsulamento

La combinazione del principio dell'astrazione dati con la tecnica dell'incapsulamento suggerisce che:

1. La rappresentazione del dato va nascosta
2. L'accesso al dato deve passare solo attraverso operazioni lecite
3. Le operazioni lecite, che ovviamente devono avere accesso alla informazione sulla rappresentazione del dato, vanno impacchettate con la rappresentazione del dato stesso

Astrazione dati & Incapsulamento

Esempio: il dato “conto corrente” ha una sua rappresentazione interna che permette di memorizzare il saldo (*balance*), il limite fido (*overcraft limit*) e il numero di conto (*account number*). La rappresentazione interna dei tre dati è nascosta. L'accesso alla rappresentazione passa per tre operazioni lecite:

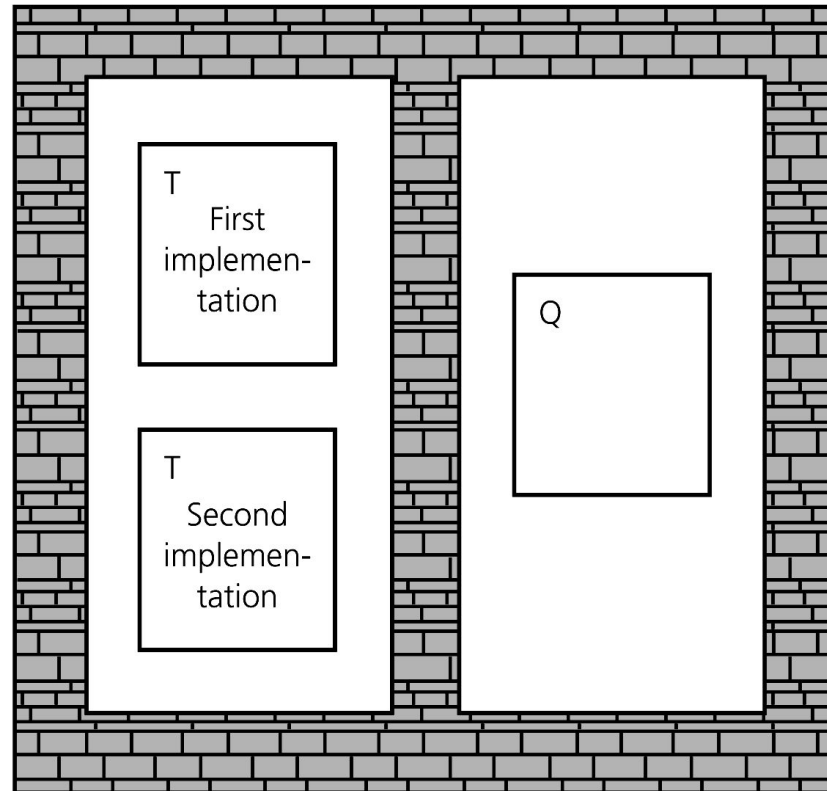


- Creazione conto
- Deposito
- Prelievo
- Stampa saldo

Rappresentazione e operazioni lecite sono impacchettate in un modulo.

Astrazione dati & Incapsulamento

T e Q sono isolati: l'implementazione di T non influenza Q



Astrazione dati vs. Astrazione funzionale

L'astrazione dati ricalca ed **estende** quella funzionale.

Attualmente la possibilità di effettuare astrazioni di dati è considerata importante almeno quanto quella di definire nuovi operatori con astrazioni funzionali. L'esperienza ha infatti dimostrato che la scelta delle strutture di dati è il primo passo sostanziale per un buon risultato dell'attività di programmazione.

Astrazione dati vs. Astrazione funzionale

L'astrazione funzionale stimola gli sforzi per evidenziare operazioni ricorrenti o comunque ben caratterizzate all'interno della soluzione di un dato problema.

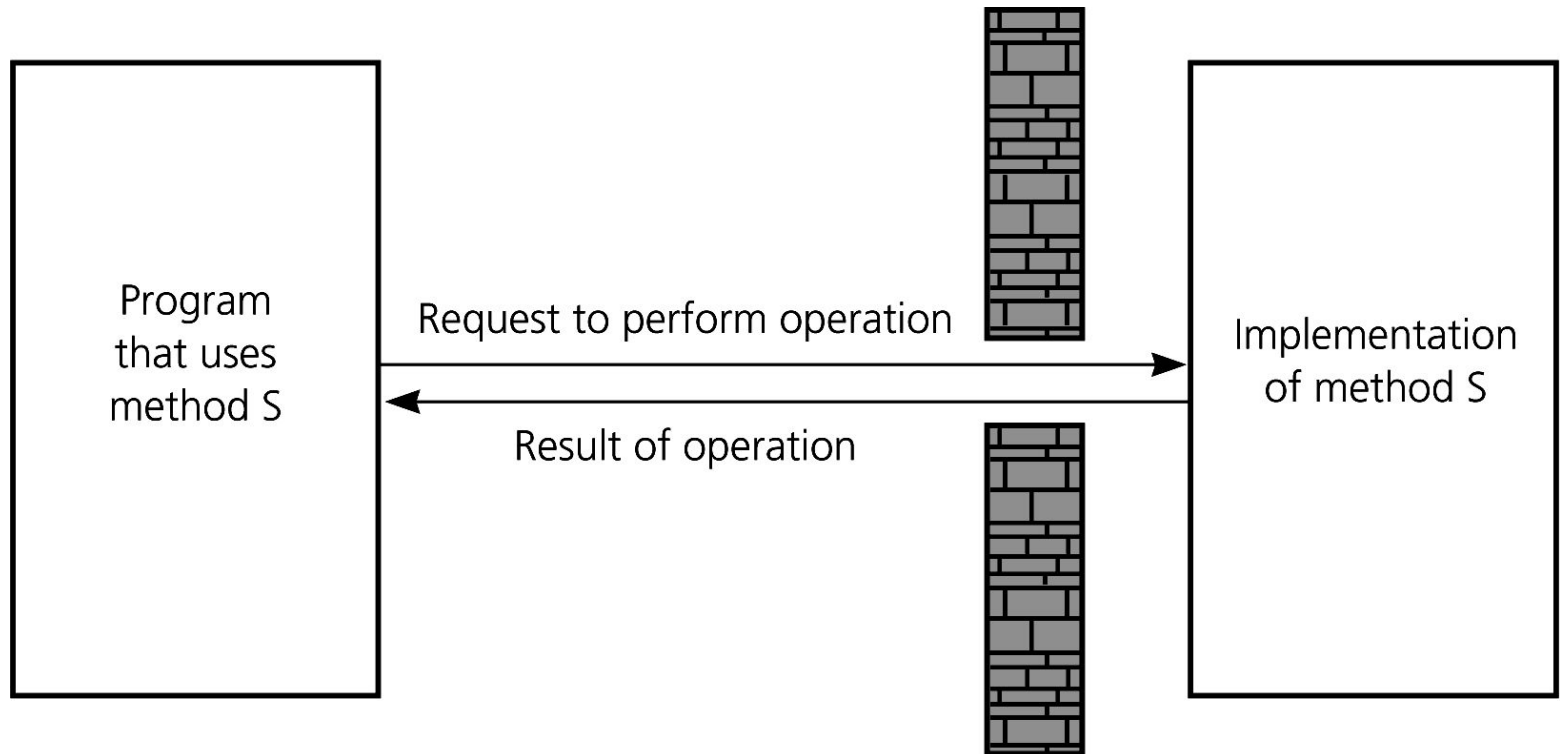
L'astrazione di dati stimola in più gli sforzi per individuare le organizzazione dei dati più consone alla soluzione del problema.

Da una progettazione *function centered* a una *data centered*.

Astrazione dati & Incapsulamento

Ovviamente, l'isolamento dei moduli non può essere totale.

- La **specifica**, o **contratto**, descrive come si può interagire con un dato astratto.



Astrazione: i punti di vista

In generale, le astrazioni supportano la separazione dei diversi interessi di

- **Utenti**: interessati a cosa si astrae (*what*)
- **Implementatori**: interessati a come (*how*) si realizza

Per questa ragione una definizione di astrazione ha sempre due componenti:

- **Specifica**
- **Realizzazione**

Per descrivere una specifica occorre ricorrere a dei *linguaggi di specifica*, che sono diversi dai linguaggi usati per descrivere le realizzazioni delle astrazioni.

Specifica sintattica e semantica

La specifica potrà poi essere:

- **Sintattica**: stabilisce quali identificatori sono associati all'astrazione
- **Semantica**: definisce il risultato della computazione inclusa nell'astrazione.

Parametrizzazione di un'astrazione

L'efficacia di un'astrazione può essere migliorata mediante l'uso di **parametri** per la comunicazione con l'ambiente esterno.

Quando un'astrazione è chiamata, ciascun **parametro formale** verrà associato in qualche modo al corrispondente **argomento**.

I meccanismi utilizzati per realizzare queste associazioni sono fondamentalmente due:

- **Meccanismi di copia**: copiano il valore da passare
- **Meccanismi definizionali**: legano direttamente il parametro formale alla definizione dell'argomento passato

Astrazione dati

Anche un'astrazione dati, come una qualunque astrazione, è costituita da una *specifica* e una *realizzazione*

- La *specifica* consente di descrivere un nuovo dato e gli operatori che ad esso sono applicabili
- La *realizzazione* stabilisce come il nuovo dato e i nuovi operatori vengono ricondotti ai dati e agli operatori già disponibili

Chi intende usare i nuovi dati con i nuovi operatori nella scrittura dei suoi programmi sarà tenuto a conoscere la specifica dell'astrazione dei dati, ma potrà astrarre dalle tecniche utilizzate per la realizzazione.

Astrazione dati

I linguaggi di specifica per astrazione dati più noti sono due:

- Il linguaggio logico-matematico usato nelle asserzioni → **specifiche assiomatiche**
- Il linguaggio dell'algebra usato nelle equazioni definite fra gli operatori specificati nel dato astratto → **specifiche algebriche.**

Astrazione dati: le specifiche assiomatiche

Un linguaggio formale per la specifica di un tipo astratto di dato è costituito dalla notazione logico-matematica delle **asserzioni**. In questo caso si parla di specifica assiomatica.

Una specifica assiomatica consta di:

- a) Una specifica **sintattica** (detta **segnatura**, *signature*) che fornisce:
 - 1) l'elenco dei NOMI dei domini e delle operazioni specifiche del tipo;
 - 2) i DOMINI di partenza e di arrivo per ogni nome di operatore;

Astrazione dati: le specifiche assiomatiche

b) Una specifica *semantica* che associa:

- 1) un INSIEME ad ogni nome di tipo introdotto nella specifica sintattica;
- 2) una FUNZIONE ad ogni nome di operatore, esplicitando le seguenti condizioni sui domini di partenza e di arrivo:
 - i) *precondizione*, che definisce quando l'operatore è applicabile
 - ii) *postcondizione*, che stabilisce la relazione tra argomenti e risultato

Il dato astratto *Vettore*

Un esempio elementare di specifica assiomatica è quella di un vettore

a) Specifica sintattica

Tipi: vettore, intero, tipoelem

Operatori:

CREAVETTORE() → vettore

LEGGIVETTORE(vettore, intero) → tipoelem

SCRIVIVETTORE(vettore, intero, tipoelem) → vettore

Il dato astratto *Vettore*

b) Specifica semantica

Tipi:

intero: l'insieme dei numeri interi

vettore: l'insieme delle sequenze di n elementi di tipo *tipoelem*

Operatori:

CREAVETTORE = v

Pre: non ci sono precondizioni, o più precisamente, il predicato che definisce la precondizione di applicabilità di CREAVETTORE è il valore VERO

*Post: $\forall i \in \{0, 1, 2, \dots, n-1\}$, l' i -esimo elemento del vettore, $v(i)$, è uguale ad un prefissato elemento di tipo *tipoelem**

LEGGIVETTORE(v, i) = e

Pre: $0 \leq i \leq n-1$

Post: $e = v(i)$

SCRIVIVETTORE(v, i, e) = v'

Pre: $0 \leq i \leq n-1$

Post: $\forall j \in \{0, 1, \dots, n-1\}, j \neq i, v'(j) = v(j), v'(i) = e$

Il dato astratto *Vettore*

c) Realizzazione

In Java il vettore (array) è un tipo di dato concreto. La corrispondenza tra la specifica appena introdotta e quella del Java è la seguente:

CREAVETTORE $\Leftrightarrow$ tipoelem v[n]

LEGGIVETTORE(v,i) $\Leftrightarrow$ v[i]

SCRIVIVETTORE(v,i,e) $\Leftrightarrow$ v[i] = e

dove *i* può essere anche un'espressione di tipo intero

Il dato astratto *Pila*

Una pila (*stack*) è una sequenza di elementi di un certo tipo in cui è possibile aggiungere o togliere elementi solo da un estremo della sequenza (la “testa”).

Può essere vista come un caso speciale di lista in cui l'ultimo elemento inserito è il primo ad essere rimosso (LIFO, *last in, first out*) e non è possibile accedere ad alcun elemento che non sia quello in testa.

Il dato astratto *Pila*

Una specifica assiomatica di una pila

Specifica sintattica

Tipi: pila, booleano, tipoelem

Operatori:

CREAPILA() $\rightarrow$ pila

PILAVUOTA(pila) $\rightarrow$ booleano

LEGGIPILA(pila) $\rightarrow$ tipoelem

FUORIPILA(pila) $\rightarrow$ pila

INPILA(tipoelem, pila) $\rightarrow$ pila

Il tipo astratto *Pila*

Specifica semantica

Tipi:

pila = insieme delle sequenze di elementi di tipo
tipoelem

booleano = insieme dei valori di verità {vero,falso}

Operatori:

CREAPILA ()=P'

Post: $P' = \wedge$, la sequenza vuota

PILAVUOTA(P)=b

Post: $b = \text{vero}$, se $P = \wedge$; $b = \text{falso}$, altrimenti

Il tipo astratto *Pila*

LEGGIPILA(P)=a

Pre: $P = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $a = a_1$

FUORIPILA(P)=P'

Pre: $P = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $P' = a_2, \dots, a_n$ se $n > 1$; $P' = \wedge$ se $n = 1$

INPILA(a,P)=P'

Pre: $P = a_1, a_2, \dots, a_n$ e $n \geq 0$

Post: $P' = a, a_1, a_2, \dots, a_n$ se $n > 0$; $P' = a$ se $n = 0$

Il dato astratto *Coda*

Una coda è un dato astratto che consente di rappresentare una sequenza di elementi in cui è possibile aggiungere elementi ad un estremo (“il fondo”) e togliere elementi dall’altro estremo (“la testa”).

Tale disciplina di accesso è detta FIFO (**First In First Out**).

È adatta a rappresentare sequenze nelle quali l’elemento viene elaborato secondo l’ordine di arrivo (lista d’attesa, etc.)

Il dato astratto *Coda*

Una specifica assiomatica di una coda

Specifica sintattica

Tipi: coda, booleano, tipoelem

Operatori:

CREACODA() $\rightarrow$ coda

CODAVUOTA(coda) $\rightarrow$ booleano

LEGGICODA(coda) $\rightarrow$ tipoelem

FUORICODA(coda) $\rightarrow$ coda

INCODA(tipoelem, coda) $\rightarrow$ coda

Il dato astratto *Coda*

Specifica semantica

Tipi:

coda = insieme delle sequenze $\langle a_1, a_2, \dots, a_n \rangle$, con $n > 0$, di elementi di tipo *tipoelem*. Λ denota la sequenza vuota.

booleano = insieme dei valori di verità {vero, falso}

Operatori:

CREACODA() $=q'$

Post: $q' = \Lambda$

CODAVUOTA(q) $=b$

Post: $b = \text{vero}$, se $q = \Lambda$; $b = \text{falso}$, altrimenti

LEGGICODA(q) $=a$

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $a = a_1$

Il dato astratto *Coda*

Specifica semantica (cont.)

FUORICODA(q)= q'

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $q' = \langle a_2, \dots, a_n \rangle$ se $n > 1$; $q' = \Lambda$ se $n = 1$

INCODA(a, q)= q'

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle$ oppure $q = \Lambda$

Post: $q' = \langle a_1, a_2, \dots, a_n, a \rangle$, se $q \neq \Lambda$;

$q' = \langle a \rangle$ se $q = \Lambda$

Esercizio

Si vuole progettare un dato astratto *Deque* (*double-ended queue* o *coda doppia*), una struttura lineare che permette di inserire, cancellare, esaminare elementi solo dalle due estremità.

Dare le specifiche assiomatiche (semantiche) di *deque*, tenuto conto delle seguenti specifiche sintattiche:

Esercizio

Tipi

deque, item, boolean

Operatori

<code>new()</code> → deque	crea nuove code doppie
<code>addq(item, deque)</code> → deque	accoda un elemento
<code>bottom(deque)</code> → item	restituisce l'elemento di coda
<code>leaveq(deque)</code> → deque	estrae l'elemento di coda
<code>push(item, deque)</code> → deque	impila un elemento
<code>top(deque)</code> → item	restituisce l'elemento di testa
<code>pop(deque)</code> → deque	estrae l'elemento di testa
<code>isnew(deque)</code> → boolean	stabilisce se una coda è vuota

Esercizio

Specifica semantica

Tipi:

deque = insieme delle sequenze $\langle a_1, a_2, \dots, a_n \rangle$, con $n > 0$, di elementi di tipo *item*. Λ denota la sequenza vuota.

boolean = insieme dei valori di verità {vero, falso}

Operatori:

new () = q'

Post: $q' = \Lambda$

Esercizio

Specifica semantica (cont.)

$\text{addq}(a, q) = q'$ // accoda un elemento

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle$ oppure $q = \Lambda$

Post: $q' = \langle a_1, a_2, \dots, a_n, a \rangle$, se $q \neq \Lambda$;
 $q' = \langle a \rangle$ se $q = \Lambda$

$\text{bottom}(q) = a$ // restituisce l'elemento di coda

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $a = a_n$

$\text{leaveq}(q) = q'$ //estrae l'elemento di coda

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $q' = \langle a_1, a_2, \dots, a_{n-1} \rangle$ se $n > 1$; $q' = \Lambda$ se $n = 1$

Esercizio

Specifica semantica (cont.)

$\text{push}(a, q) = q'$ // impila un elemento

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle$ oppure $q = \Lambda$

Post: $q' = \langle a, a_1, a_2, \dots, a_n \rangle$, se $q \neq \Lambda$;
 $q' = \langle a \rangle$ se $q = \Lambda$

$\text{top}(q) = a$ // restituisce l'elemento di testa

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $a = a_1$

$\text{pop}(q) = q'$ // estrae l'elemento di testa

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $q' = \langle a_2, \dots, a_n \rangle$ se $n > 1$; $q' = \Lambda$ se $n = 1$

Esercizio

Specifica semantica (cont.)

isnew(q)=b // stabilisce se una coda doppia è vuota

Post: b=vero, se $q=\wedge$; b=falso, altrimenti

Esercizio

Si vuole progettare il dato astratto *volo*.

Dare le specifiche assiomatiche (semantiche) di *volo*, tenuto conto delle seguenti specifiche sintattiche

Esercizio

Tipi

volo, passeggero, posto, integer, boolean

Operatori

creavolo() → volo crea un nuovo volo

postolibero(posto, volo) → boolean stabilisce se il posto è libero
o meno

check-in(posto, passeggero, volo) → volo aggiunge un passeggero
al volo assegnandogli il posto specificato, purché libero

libera(posto, volo) → volo libera un posto sul volo

sostituisci(posto, passeggero, volo) → volo assegna a un altro
passeggero un posto già occupato

passeggero(posto, volo) → passeggero restituisce il passeggero
che occupa un posto

passeggeri(volo) → integer restituisce il numero di passeggeri
con carta di imbarco, cioè che hanno fatto il check-in

Esercizio

Suggerimento:

posto è l'insieme K dei posti

passaggero è l'insieme P dei passeggeri.

volo è l'insieme di voli, dove ogni volo è a sua volta un insieme di coppie $(k, p) \in K \times P$.

integer è l'insieme Z dei numeri interi. 0 e 1 sono particolari valori dell'insieme Z .

boolean è l'insieme dei valori di verità $\{\text{true}, \text{false}\}$.

N.B.: Questa definizione è sufficientemente astratta e non dice nulla sull'insieme K . Infatti i posti su un aereo sono generalmente identificati da una coppia

Numero Lettera

Esercizio

Specifica semantica

Tipi:

volo = insieme di coppie $(k,p) \in K \times P$ dove K è l'insieme dei posti e P è l'insieme dei passeggeri. $\emptyset$ denota l'insieme vuoto.

integer = insieme di numeri naturali

boolean = insieme dei valori di verità {vero, falso}

Operatori:

creavolo() = v' // crea un nuovo volo

Post: $v' = \emptyset$

Esercizio

Specifica semantica (cont.)

postolibero(k, v) = b // stabilisce se il posto è libero o meno

Pre: $v \subseteq K \times P, k \in K$

Post: $b = \text{vero}$ se $\neg(\exists p \in P (k, p) \in v)$, falso altrimenti

check-in(k, p, v) = v' // aggiunge un passeggero al volo
assegnandogli il posto specificato, purché libero

Pre: $v \subseteq K \times P, k \in K, p \in P \neg(\exists q \in P (k, q) \in v)$,

Post: $v' = v \cup \{(k, p)\}$

Esercizio

Specifica semantica (cont.)

$\text{libera}(k, v) = v'$ // libera un posto sul volo

Pre: $v \subseteq K \times P, k \in K, \exists q \in P (k, q) \in v$

Post: $v' = v - \{(k, q)\}$

$\text{sostituisci}(k, p, v) = v'$ // assegna a un altro passeggero un posto già
occupato

Pre: $v \subseteq K \times P, k \in K, p \in P, \exists q \in P (k, q) \in v$

Post: $v' = (v - \{(k, q)\}) \cup \{(k, p)\}$

Esercizio

Specifica semantica (cont.)

passaggero(k, v)=p //restituisce il passeggero che occupa un posto

Pre: $v \subseteq K \times P$, $k \in K$, $\exists q \in P (k, q) \in v$

Post: $p=q$

passaggeri(v)=n //restituisce il numero di passeggeri con carta di imbarco, cioè che hanno fatto il check-in

Pre: $v \subseteq K \times P$

Post: $n=|v|$

Limiti delle specifiche assiomatiche

Si noti che il metodo di specifica assiomatica è preciso nella definizione della specifica sintattica, mentre **è piuttosto informale per gli altri aspetti** (talvolta si ricorre al linguaggio naturale per semplicità). Pertanto esso non consente di caratterizzare precisamente un tipo astratto.

In particolare non consente di definire i valori che possono essere generati mediante l'applicazione di operatori, e non consente di stabilire quando l'applicazione di diverse sequenze di operatori porta al medesimo valore.

Questo problema è superato dalle specifiche algebriche.

Dato Astratto: specifiche algebriche

Le specifiche algebriche si basano sull'algebra, piuttosto che sulla logica. Essenzialmente definiscono un dato astratto come un'algebra eterogenea, ovvero come una collezione di diversi insiemi su cui sono definite diverse operazioni.

Le algebre tradizionali sono omogenee. Un'algebra omogenea consiste in un unico insieme e diverse operazioni.

Esempio: $\mathbb{Z}$ con le operazioni di addizione e moltiplicazione è un'algebra omogenea.

Dato Astratto: specifiche algebriche

Esempio: dato un alfabeto Σ , indichiamo con Σ^* l'insieme di tutte le stringhe, incluso quella vuota, costruite con i simboli di Σ .

Σ^* con le operazioni di concatenazione e calcolo della lunghezza non sono un'algebra omogenea, visto che il codominio dell'operazione di calcolo della lunghezza è $\mathbb{N}$ e non Σ^* .

Quindi quest'algebra consiste di due insiemi, stringhe e interi, su cui sono definite le operazioni di concatenamento e lunghezza delle stringhe.

Dato Astratto: specifiche algebriche

Una specifica algebrica consiste di tre parti:

1. **Sintattica:** elenca i nomi del tipo, le sue operazioni e il tipo degli argomenti delle operazioni. Se un'operazione è una funzione allora è specificato il codominio (range) della funzione.
2. **Semantica:** consiste di un insieme di equazioni algebriche che descrivono in modo indipendente dalla rappresentazione le proprietà delle operazioni.
3. **Di restrizione:** stabilisce varie condizioni che devono essere soddisfatte o prima che siano applicate le operazioni o dopo che esse siano state completate.

N.B.: alcuni autori inglobano le specifiche di restrizione in quelle semantiche.

Dato Astratto: specifiche algebriche

Uno degli aspetti più interessanti delle specifiche algebriche è la semplicità del linguaggio di specifica rispetto ai linguaggi di programmazione procedurale.

Infatti, il linguaggio di specifica consiste di solo cinque primitive:

1. composizione funzionale
2. relazione di eguaglianza
3. 4. due costanti, **true** e **false**
5. un numero illimitato di variabili libere

Dato Astratto: specifiche algebriche

La funzione *if then else* può essere facilmente descritta dalle seguenti equazioni:

$$\textit{if then else} (\textit{true}, q, r) = q$$

$$\textit{if then else} (\textit{false}, q, r) = r$$

Questa funzione è così importante che si assume già data come operatore infisso:

$$\textit{if } p \textit{ then } q \textit{ else } r$$

Inoltre si assume che sono predefiniti i valori interi e booleani.

Specifica algebrica di *Pila*

Specifica sintattica

sorts: stack, item, boolean

operations:

newstack() -> stack

push(stack, item) -> stack

pop(stack) -> stack

top(stack) -> item

isnew(stack) -> boolean

Ogni insieme è detto un **sort** (letteralmente “tipo”) dell'algebra eterogenea.

I sort **item** e **boolean** sono **ausiliari** alla definizione di stack.

Specifica algebrica di *Pila*

Specifica semantica

declare **stk**: stack, **i**: item

pop(push(stk, i)) = stk

top(push(stk, i)) = i

isnew(newstack) = true

isnew(push(stk, i)) = false

Nella specifica semantica
occorre **dichiarare** i
parametri su cui lavorano
gli operatori.

Specifica algebrica di *Pila*

Specifica di restrizione

restrictions

pop(newstack) = error

top(newstack) = error

dove 'error' è un elemento speciale indefinito.

Ossiomatrica vs. Algebrica

Ossiomatrica: *"Se hai il serbatoio pieno (**Pre**) e giri la chiave (**Operazione**), allora il motore si accende (**Post**)"*. È una visione **imperativa** e legata allo **stato**.

Algebrica: Non ci importa come è fatto il motore dentro. Ci importa che *"Accelerare + Frenare = Restare dove sei"*. È una visione **dichiarativa** e legata alle **proprietà**.

In pratica, stiamo definendo un Tipo di Dato Astratto come se fosse un sistema di equazioni matematiche.

Specifica algebrica

I **tre** pilastri della specifica algebrica sono:

SORT	AZIONI (specifica sintattica)	EQUAZIONI (specifica semantica)
Quali tipi di dati stiamo usando? (es. stack, item, boolean)	La firma delle funzioni. Nome funzione, argomenti di input e di output.	Le regole del gioco, qui scriviamo le verità attraverso l'utilizzo di equazioni.

Specifica algebrica di *Pila*

Ovviamente avremmo potuto scrivere molte altre equazioni, come:

$$isnew(pop(push(newstack, i))) = true$$

che evidenzia come il predicato *isnew* sia vero anche per quegli stack ottenuti inserendo un generico elemento *i* in un **nuovo** stack e poi rimuovendolo.

Tuttavia questa equazione è **ridondante**, poiché è ricavabile dalle altre scritte in precedenza. Infatti, grazie alla prima e terza equazione possiamo scrivere:

$$isnew(pop(push(newstack, i))) = isnew(newstack) = true$$

Specifica algebrica di *Pila*

Nelle specifiche semantiche è importante indicare l'insieme minimale di equazioni (dette **assiomi**) a partire dalle quali possiamo derivare tutte le altre.

Le specifiche semantiche si diranno **incomplete** se non permetteranno di derivare tutte le verità (equazioni) desiderate dell'algebra specificata.

Le specifiche semantiche si diranno **inconsistenti** (o contraddittorie) se permetteranno di derivare delle equazioni indesiderate, cioè considerate false.

Le specifiche semantiche si diranno **ridondanti** se alcune delle equazioni sono ricavabili dalle altre.

Costruttori e Osservazioni

Scrivere delle specifiche semantiche complete, consistenti e non ridondanti può non essere un compito semplice.

Per questo conviene introdurre una **metodologia**, che si basa sulla distinzione degli operatori di un dato astratto in:

- **Costruttori**, che creano o istanziano il dato astratto
- **Osservazioni**, che ritrovano informazioni sul dato astratto

Il comportamento di una astrazione dati può essere specificata riportando il valore di **ciascuna osservazione applicata a ciascun costruttore**.

Questa informazione è organizzata in modo naturale in una matrice, con i costruttori lungo una dimensione e le osservazioni lungo l'altra.

Costruttori e Osservazioni

<i>osservazioni</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>pop(stk')</i>	error	stk
<i>top(stk')</i>	error	i
<i>isnew(stk')</i>	true	false

Osservazioni binarie

Tutte le osservazioni viste finora sono unarie, nel senso che esse osservano un singolo valore del dato astratto.

Spesso è necessario disporre di osservazioni più complesse.

Per confrontare due valori è necessario osservare due istanze del dato astratto.

Questo complica la specifica, perché il valore dell'osservazione dev'essere definito per tutte le **combinazioni di costruttori** possibili per i valori astratti che si devono confrontare.

Osservazione binaria: $\text{equal}(l, m)$

Esempio: predicato $\text{equal}(l, m)$ che è vero solo se le due pile contengono gli stessi elementi nello stesso ordine.

<i>Costruttore di m</i>	<i>Costruttore di l</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>newstack</i>	true	false
<i>push(stk', i')</i>	false	$i=i'$ and $\text{equal}(\text{stk}, \text{stk}')$

Questa tabella può essere vista come l'aggiunta di una terza dimensione alla tabella di base per le osservazioni unarie.

Osservazione binaria: $\text{equal}(\text{stk}', m)$

Questa terza dimensione può essere rimossa andando a classificare esplicitamente solo il primo argomento dell'osservazione e referenziando in modo astratto il secondo argomento mediante una **variabile libera**.

<i>osservazione</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>equal(stk', m)</i>	isnew(m)	not isnew(m) and i=top(m) and equal(stk, pop(m))

Specifica algebrica di coda

Specifica sintattica

sorts: queue, item, boolean

operations:

newq() $\rightarrow$ queue

addq(queue, item) $\rightarrow$ queue

deleteq(queue) $\rightarrow$ queue

frontq(queue) $\rightarrow$ item

isnewq(queue) $\rightarrow$ boolean

Richiamo: Specifica assiomatica di *Coda*

Specifica semantica

Operatori:

CREACODA() $=Q'$

Post: $Q' = \wedge$, la sequenza vuota

CODAVUOTA(Q) $=b$

Post: $b = \text{vero}$, se $Q = \wedge$; $b = \text{falso}$, altrimenti

LEGGICODA(Q) $=a$

Pre: $Q = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $a = a_1$

FUORICODA(Q) $=Q'$

Pre: $Q = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $Q' = a_2, \dots, a_n$ se $n > 1$; $Q' = \wedge$ se $n = 1$

INCODA(a,Q) $=Q'$

Pre: $Q = a_1, a_2, \dots, a_n$ e $n \geq 0$

Post: $Q' = a_1, a_2, \dots, a_n, a$ se $n > 0$; $Q' = a$ se $n = 0$

Costruttori e Osservazioni

<i>osservazioni</i>	<i>Costruttore di q'</i>	
	<i>newq</i>	<i>addq(q, i)</i>
<i>isnew(q')</i>	true	false
<i>frontq(q')</i>	error	if isnewq(q) then i else frontq(q)
<i>deleteq(q')</i>	error	if isnewq(q) then newq else addq(delete(q), i)

Specifica algebrica di coda

Specifica semantica

declare **q**, **r**: queue, **i**: item

isnewq(newq) = true

isnewq(addq(q, i)) = false

deleteq(addq(q, i)) = **if** isnewq(q) **then** newq **else** addq(deleteq(q), i)

frontq(addq(q, i)) = **if** isnewq(q) **then** i **else** frontq(q)

Specifica algebrica di coda

Specifica di restrizione

restrictions

frontq(newq) = error

deleteq(newq) = error

Esercizio

- Fornire una specifica algebrica completa, consistente e minimale (cioè non ridondante) del dato astratto *Vettore* di cui è stata fornita la specifica assiomatica mediante pre- e postcondizioni.

sorts: tipoelem, array, integer

operations:

new() -> array

read(array, integer) -> tipoelem

write(array, integer, tipoelem) -> array

N.B. Supponiamo che l'array sia inizializzato con il valore 0.

Esercizio (cont.)

<i>osservazioni</i>	<i>Costruttore di v'</i>	
	<i>new()</i>	<i>write(v, i, e)</i>
<i>read(v', i')</i>	0	if i=i' then e else read(v, i')

Specifica semantica

declare: **i, j**: integer; **e**: tipoelem; **v**: array

read(new()) = 0

read(write(v, i, e), j) = **if** i=j **then** e **else** read(v, j)

Esercizio (cont.)

Miglioriamo la specifica di vettore con l'introduzione del controllo sugli indici.

Specifica sintattica

sorts: array, integer, tipoelem

operations:

create(integer, integer) -> array //crea un vettore con lower/upper bound

assign(array, integer, tipoelem) -> array

first(array) -> integer // restituisce lower bound

last(array) -> integer // restituisce upper bound

eval(array, integer) -> tipoelem // restituisce l'elemento

Esercizio (cont.)

<i>osservazioni</i>	<i>Costruttore di v'</i>	
	<i>create(x, y)</i>	<i>assign(v, i, e)</i>
<i>first(v')</i>	x	<i>first(v)</i>
<i>last(v')</i>	y	<i>last(v)</i>
<i>eval(v', i')</i>	error	<i>if $i' < \text{first}(a)$ or $i' > \text{last}(a)$ then error else if $i' = i$ then e else eval(v, i')</i>

Esercizio (cont.)

Specifica semantica

declare: **x, y, i, j**: integer; **e**: tipoelem; **v**: array

$\text{first}(\text{create}(x, y)) = x$

$\text{first}(\text{assign}(v, i, e)) = \text{first}(v)$

$\text{last}(\text{create}(x, y)) = y$

$\text{last}(\text{assign}(v, i, e)) = \text{last}(v)$

$\text{eval}(\text{create}(x, y), i) = \mathbf{error}$

$\text{eval}(\text{assign}(v, i, e), j) = \mathbf{if } j < \text{first}(v) \text{ or } j > \text{last}(v) \text{ then } \mathbf{error} \text{ else if } i = j \text{ then } e \text{ else}$

$\text{eval}(v, j)$

Esercizio

- Fornire una specifica algebrica semantica completa, consistente e minimale per il dato astratto *Stringa* supposto che le specifiche sintattiche siano le seguenti:

sorts: string, char, integer, boolean

operations:

new() → string crea nuove stringhe

append(string, string) → string concatena due stringhe

add(string, char) → string aggiunge un carattere a fine stringa

length(string) → integer calcola la lunghezza di una stringa

isEmpty(string) → boolean stabilisce se la stringa è vuota

equal(string, string) → boolean stabilisce se due stringhe sono uguali

La scelta dei costruttori

Talvolta, l'applicazione della metodologia per la sintesi di specifiche algebriche può comportare delle difficoltà riguardo al **discernimento di cosa è costruttore e cosa operazione**.

Ad esempio, guardando la specifica sintattica del dato astratto *Stringa* possiamo dire che tutti gli operatori che restituiscono una stringa sono dei candidati a essere dei costruttori.

`new()` -> string

`append(string, string)` -> string

`add(string, char)` -> string

Mentre non ci sono dubbi sul primo, restano delle perplessità sul fatto che entrambi gli altri operatori debbano essere dei costruttori.

La scelta dei costruttori

Nello scegliere i costruttori adotteremo il **criterio di minimalità**,
cioè l'insieme dei costruttori dev'essere il più piccolo insieme di
operatori necessario a costruire tutti i possibili valori per un certo
dato astratto.

Ora, è evidente che per costruire una stringa abbiamo bisogno di:

`new() -> string` `add(string, char) -> string`

Il terzo operatore:

`append(string, string) -> string`

non è necessario, quindi **non** lo scegliamo come costruttore.

Un criterio euristico: *gli argomenti di un costruttore non devono
essere tutti dello stesso sort del dato astratto.*

Esercizio (cont.)

<i>osservazioni</i>	<i>Costruttore di s'</i>	
	<i>new()</i>	<i>add(s, c)</i>
<i>length(s')</i>	0	length(s)+1
<i>isEmpty(s')</i>	true	false
<i>append(t,s')</i>	t	add(append(t,s),c)

Esercizio (cont.)

`equal(string, string)` ☐ boolean

<i>Costruttore di t'</i>	<i>Costruttore di s'</i>	
	<i>new()</i>	<i>add(s, c)</i>
<i>new()</i>	true	false
<i>add(t, h)</i>	false	if h=c then equal(t, s) else false

Esercizio

Progettare il dato astratto *Conto Con Fido* che consente di rappresentare dei conti correnti bancari per i quali è permesso uno scoperto. Si deve poter limitare lo scoperto tramite concessione del fido. Le operazioni ammesse per questo dato astratto sono:

conto (account): apre un nuovo conto corrente bancario definendo saldo iniziale e massimo scoperto ammesso

saldo (balance): riporta il saldo del conto

deposita (deposit): deposita una somma sul conto

preleva (withdraw): preleva denaro dal conto

concediFido (setOverdraftLimit): definisce il limite massimo dello scoperto

fidoConcesso (getOverdraftLimit): restituisce il limite massimo dello scoperto

Esercizio (cont.)

Specifica sintattica

sorts: contocorrente, saldo, denaro, scoperto;

operations:

conto(saldo, scoperto) -> contocorrente

saldo(contocorrente) -> saldo

deposita(contocorrente, denaro) -> contocorrente

preleva(contocorrente, denaro) -> contocorrente

concediFido(contocorrente, scoperto) -> contocorrente

fidoConcesso(contocorrente) -> scoperto

Esercizio (cont.)

Specifica semantica

Il dominio *contocorrente* è l'insieme delle coppie $(s, l) \in \mathbf{R} \times \mathbf{R}^+$, tali che $s \geq -l$. Il dominio *saldo* corrisponde all'insieme dei numeri reali $\mathbf{R}$, mentre i domini *scoperto* e *denaro* corrispondono ad $\mathbf{R}^+$.

Pertanto potremo disporre sui vari domini di tipiche operazioni algebriche, e in particolare:

- due operazioni binarie: “+” e “-”
- operatore unario: “-”

nonché di una relazione binaria “<” e di una costante “0”.

N.B.: formalmente avremmo dovuto indicare che “+” è definito sui domini *saldo* e *scoperto* e restituisce un *saldo*, etc.,

Esercizio (cont.)

N.B.: Gli operatori aggiuntivi che sono stati specificati non fanno riferimento al dominio *contocorrente* relativo al dato astratto che si vuole definire. Per questo non è necessario specificarli.

Diversamente, se fosse stata ipotizzata la disponibilità di un operatore del dominio *contocorrente*, avremmo dovuto successivamente indicarne la semantica al pari degli altri operatori indicati nella specifica sintattica.

Esercizio (cont.)

Possiamo identificare un solo costruttore (*conto*). Gli altri operatori sono tutte osservazioni.

<i>osservazioni</i>	<i>Costruttore di c</i>
	<i>conto(s,l)</i>
<i>saldo(c)</i>	s
<i>deposita(c,d')</i>	conto(s+d',l)
<i>preleva(c,d')</i>	if s-d'<-l then error else conto(s-d',l)
<i>concediFido(c,l')</i>	if s<-l' then error else conto(s,l')
<i>fidoConcesso(c)</i>	1

Esercizio (cont.)

Specifica semantica

declare **c**: contocorrente, **s**: saldo, **d**: denaro, **l**, **l'**: scoperto;

saldo(conto(s,l))=s

deposita(conto(s,l), d) = conto(s+d,l)

preleva(conto(s,l), d) = **if** s-d<-l **then** error **else** conto(s-d,l)

concediFido(conto(s,l),l') = **if** s<-l' **then** error **else** conto(s,l')

fidoConcesso(conto(s,l))=l

Esercizio (cont.)

Utilizzando le precedenti specifiche è possibile dimostrare la seguente equazione:

$$\text{preleva}(\text{deposita}(c, d), d) = c$$

Dim.: Sia $c = \text{conto}(s, l)$

$$\text{preleva}(\text{deposita}(\text{conto}(s, l), d), d) = \text{preleva}(\text{conto}(s+d, l), d)$$

per definizione di *deposita* ($\text{deposita}(\text{conto}(s, l), d) = \text{conto}(s+d, l)$)

Inoltre:

$$\text{preleva}(\text{conto}(s+d, l), d) = \text{conto}((s+d)-d, l)$$

per definizione di *preleva* ($\text{preleva}(\text{conto}(s, l), d) = \text{if } s-d < -l \text{ then error else } \text{conto}(s-d, l)$).

$$\text{Ma } \text{conto}((s+d)-d, l) = \text{conto}(s, l) = c$$

Astrazione di controllo

- L'astrazione di controllo si riferisce alla possibilità di specificare un modulo software che **esegue** delle operazioni in un **ordine**, nascondendo i dettagli su come il mantenimento dell'ordine è ottenuto
- In sintesi:
 - Il modulo software deve essere parametrizzato rispetto alle operazioni da eseguire
 - Il modulo software è associato a un controllo di sequenza (*control flow*)
 - I dettagli di come il controllo di sequenza è garantito non sono visibili al consumatore (fruitore) del modulo

Astrazione di controllo

- Un modulo che ha in input un'espressione e un comando e itera l'esecuzione del comando fintanto che l'espressione è vera, è un modulo che implementa il costrutto di controllo `while`
- L'effettiva implementazione del `while` è nascosta

Esempio: `while(espressione, istruzione)`

Può essere realizzato iterativamente in vari modi:

1. Valuta espressione
2. Se true allora vai a 4
3. Esci dal modulo (return)
4. Esegui istruzione
5. Vai a 1

1. Valuta espressione
2. Se false allora vai a 5
3. Esegui istruzione
4. Vai a 1
5. Esci dal modulo (return)

Astrazione di controllo

Esempio (cont.): o persino ricorsivamente!

1. Valuta espressione
2. Se true allora
 - a) Esegui istruzione
 - b) `while(espressione, istruzione).`
3. Esci dal modulo (`return`)

Tutte queste realizzazioni (iterative o ricorsive) sono corrette rispetto alla semantica dell'astrazione di controllo `while`. All'utente del modulo non importa sapere quale delle realizzazioni è stata adottata.

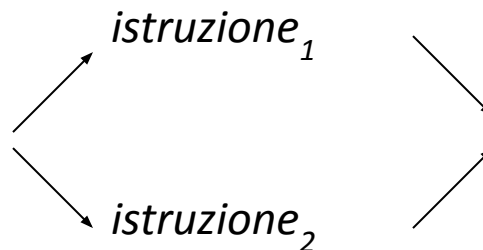
Astrazione di controllo

Nella programmazione sequenziale l'ordine di esecuzione delle istruzioni è **totale**. Nella programmazione concorrente/parallela l'ordine di esecuzione può essere **parziale**.

Esempio: l'astrazione di controllo

fork-join(istruzione₁, istruzione₂)

che prevede l'esecuzione parallela di *istruzione₁* e *istruzione₂* definisce una relazione d'ordine parziale fra le due istruzioni.



Astrazione di controllo

I linguaggi di specifica utilizzati per l'astrazione di controllo si basano sulla definizione di **relazioni di precedenza** fra istruzioni che permettono di stabilire sia ordinamenti totali che ordinamenti parziali.

La combinazione dell'astrazione dati con l'astrazione di controllo permette di sviluppare soluzioni software **invarianti** non solo al cambiamento della realizzazione dei dati ma anche **al cambiamento dei meccanismi di visita del dato astratto**.

Ciò risulta molto utile poiché l'iterazione su collezioni di dati è un compito comune e ripetitivo e l'evidenza empirica indica che spesso si commettono errori nella inizializzazione dell'iterazione e nella scrittura delle condizioni di stop.

Astrazione di controllo

Esempio: Un dato astratto che **contiene** un insieme di valori può mettere a disposizione due operatori, **next** che restituisce un valore e il predicato **hasNext** che è vero se ci sono ancora altri valori da selezionare. L'astrazione di controllo *for-each(dato astratto, istruzione)* utilizzerà questi due operatori per eseguire l'*istruzione* su ogni valore contenuto nel dato astratto. **L'informazione sull'ordine con cui i valori sono considerati (cioè sul protocollo di iterazione) è nascosta all'utente del *for-each*.**

Andando ancora oltre, si può ipotizzare che il dato astratto deleghi un modulo **iteratore** a fornire i servizi richiesti dal *for-each*. In questo modo i meccanismi di visita sono tutti incapsulati in un modulo.

Ancora una volta, la tecnica dell'incapsulamento viene in aiuto al progettista che vuole applicare una qualche forma di astrazione.

Riferimenti bibliografici

Per la progettazione con astrazione funzionale e astrazione dati

Peter Klein

Designing Software with Modula-3

Per le specifiche assiomatiche di vettore, pila e coda

Alan Bertossi

Algoritmi e Strutture di Dati

UTET, 2000

Capitoli 0, 1, 4

Per le specifiche algebriche

John D. Gannon, James M. Purtilo, Marvin V. Zelkowitz

Software Specification: A Comparison of Formal Methods

2001

Capitolo 5.3

Riferimenti bibliografici

Per le specifiche algebriche si può fare riferimento anche al testo:

A. Fuggetta, C. Ghezzi, S. Morasca, A. Morzenti, M. Pezzè

Ingegneria del software: progettazione, sviluppo e verifica

Mondadori Informatica, 1991

Capitolo 4.3